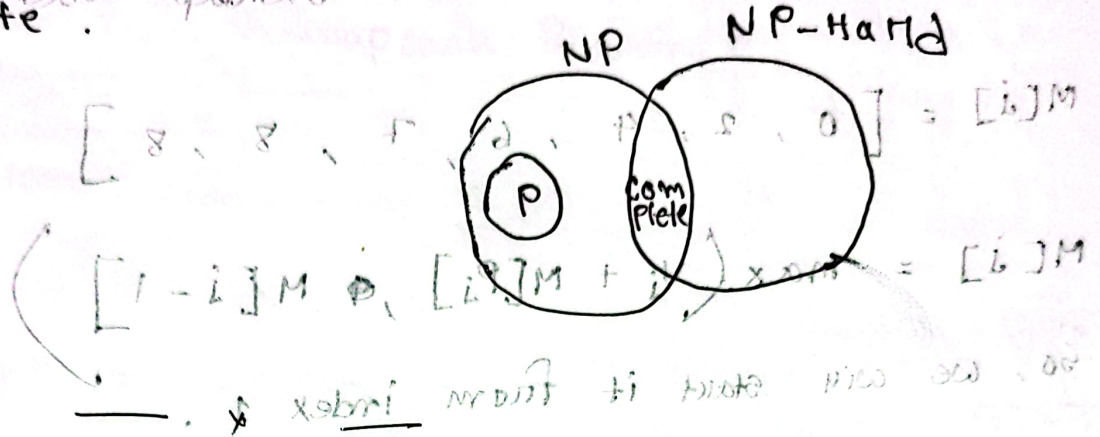


NP-Hard $\rightarrow$ If it is a problem is NP-hard if all problems in NP are polynomial time reducible to it even though it may not be NP itself.

NP-~~Hard~~ ^{complete} $\rightarrow$ if NP-hard uses non deterministic time algorithm to solve itself then it is known as NP-complete.

* $P \subseteq NP$



Memorization $\rightarrow$ Top-down

Dynamic programming $\rightarrow$ Bottom-up

Weighted interval Scheduling

1. First you have to sort it by end time.

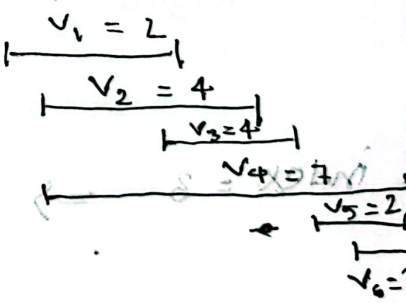
2. Then you have to track weight in a list

$V_1 = [0, 2, 4, 4, 7, 8, 1]$

3. you have to track who ends before the current work =

$P = [0, 0, 0, 1, 0, 3, 3]$

যদিও প্রথম ভিন্নতর value সবার সবার মধ্যে overlap
করে তবে পরের previous value 0 তৈরি,



* 3 స్వల్ప index, 5- value లకు వర్తించే (అధిక) < 256 bit

index 1, 5- value

* - Δt_{avg} - Δt_{avg} previous value 3 output - Δt_{avg} 3277 - 94
 index in Δt_{avg} value - 21 overlap - Δt_{avg} 3277 - 94 60

$$M[i] = \max(V_i + M[p_i], M[i-1])$$

So, we will start it from i = 1

index = 1 $\Rightarrow \max(2 + 0, 0) \leftarrow$ normalization #
 $= 2$ $\downarrow$ $0 \leftarrow$ previous value, cause we want maximum.
 Original value 2.

index = 2 $\xrightarrow{+2i}$ max (4 + 0, 0, 2, 8, 7, 4, 5, 0) = 14

index = 3 $\Rightarrow$ $\max(4 + 2, 4) = 6$

[$\rightarrow$ 1 number, index, 20] value

index = 4 $\Rightarrow \max(6+2, 7) = 8$

index = 5 $\Rightarrow \max(6+1, 8) = 8$

that is And the max would be 8

	0	1	2	3	4	5	6	7	8	9
W	0	1	2	3	4	5	6	7	8	9
V	0	1	2	3	4	5	6	7	8	9
# knapsack Problem #	0	1	2	3	4	5	6	7	8	9

Knapsack problem can take fraction of a value.

প্রথমে আমরা দেখব, কত Value weight - মানে per weight value - তার মধ্যে থেকে, তারপর ওই প্রস্তুত করে নিতে হবে, then কোন value নেই পূরণ না, নিতে পারলে হ্যাঁ ঠিক নিতে পারবে।

1. 0/1 Knapsack Problem #

এখানে কোন value কে fraction হিসাবে নেওয়া পারেনা, নিতে পারেনা - অর্থাৎ 0/1 - অর্থাৎ 0 বা 1 - অর্থাৎ 0/1 Knapsack Problem #

weight = {2, 3, 4, 5}

Item - 7. (10/12)

যাচ্ছে হার্ড-ড্রাকটে দিয়ে-সাজানো

১৩৩১ - ১৩৩২

LCS

Longest common subsequence.

String 1 : a b c d e f g h i j

String 2 : e c d g i

* ଧାରଣା ଯାହା ଯୋଗାଯୋଗ, ଏହା ଏହା ଧାରଣା ଯୋଗାଯୋଗ ଉପରେ କାର୍ଯ୍ୟ କରେ ।

code →

LCS (i, j)

i → ^{index} Value of first string

j → ^{index} Value of second "

if (A[i] == None or B[j] == None):

return 0

elif A[i] == B[j]:

return 1 + LCS(i+1, j+1)

else:

return max(LCS(i+1, j), LCS(i, j+1))

A [b | d | None]

B [a | b | c | d | None]

Time complexity → $m \times n$ → Top down.

dynamic code

if (A[i] == B[j]):

LCS[i, j] = 1 + LCS[i-1, j-1]

else:

LCS[i, j] = max(LCS[i-1, j], LCS[i, j-1])

→ Bottom to top